Name: Cohort/Batch: Date: Score:

NESTJS PRACTICE SHEET - 10 CONCEPTUAL Q&A

10 Q&A on modules, DI, guards, pipes, interceptors, and testing

TOTAL: 10 QUESTIONS

Q1 How does NestJS's modular architecture work? — Threat Model

Q2 How does NestJS's dependency injection container work? — Architecture

Q3 What are NestJS decorators and how do they work? — Configuration

Q4 How do NestJS Pipes work and how do you validate request bodies? — Hardening

Q5 What are NestJS Guards and how do you implement RBAC? — SSO / IdP

Q6 What is a NestJS exception filter and how does it standardize error responses? — Observability

Q7 What are NestJS interceptors used for? — Lifecycle

Q8 How do you integrate TypeORM with NestJS? — Compliance

Q9 How does NestJS support microservice communication? — Testing

Q10 How do you write unit tests in NestJS? — Tuning

ANSWER KEY & EXPLANATORY SOLUTIONS

Comprehensive verified answers, best-practice configs, security controls & reference

VERIFIED SOLUTIONS

A1 A Module groups related controllers and

Mitigation

A Module groups related controllers and providers. @Module({imports, controllers, providers, exports}) defines its boundaries. The AppModule is the root. Feature modules import other modules and export providers they want to share with importers.

A2 Providers declared in a module's providers

Primitives

Providers declared in a module's providers array are added to the DI container. NestJS resolves constructor injection by type token. Provider scope can be DEFAULT (singleton per module), REQUEST (new instance per request), or TRANSIENT (new per injection).

A3 Decorators are TypeScript metadata annotations.

Hardening

Decorators are TypeScript metadata annotations. @Get('/path') stores route metadata on the method. @Body(), @Param(), @Query() bind request data. NestJS's RouterExplorer reads this metadata at startup to register Express/Fastify routes automatically.

A4 Pipes transform or validate data before

Gotchas

Pipes transform or validate data before it reaches the handler. Global ValidationPipe with whitelist:true strips undeclared properties and transforms payloads. class-validator decorators (@IsEmail(), @MinLength(8)) on DTO classes define validation rules.

A5 A Guard implements CanActivate. Return true

Federation

A Guard implements CanActivate. Return true to allow or false to deny. @UseGuards(RolesGuard) applies it to a controller or method. The guard reads @Roles('admin') metadata via Reflector and checks the authenticated user's roles against the required roles.

A6 Implement ExceptionFilter, annotate with

Observability

Implement ExceptionFilter, annotate with @Catch(MyException), and define catch(exception, host). It intercepts thrown exceptions and returns a structured JSON error response. Register globally with app.useGlobalFilters(new MyFilter()) for consistent API error shapes.

A7 Interceptors wrap request handling before and

Automation

Interceptors wrap request handling before and after. The intercept(ctx, next) method calls next.handle() which returns an Observable. Use RxJS operators to transform the response (map), measure execution time, add caching, or log requests and responses.

A8 Import TypeOrmModule.forRoot(options) in

Governance

Import TypeOrmModule.forRoot(options) in AppModule. Import TypeOrmModule.forFeature([UserEntity]) in the feature module to get a repository. Inject InjectRepository(UserEntity) private userRepo: Repository<UserEntity> into services.

A9 NestJS microservices use ClientProxy to send

Assessment

NestJS microservices use ClientProxy to send messages and emit events. Transport strategies include TCP, Redis, NATS, Kafka, and RabbitMQ. @MessagePattern('pattern') on a handler receives messages. Both request-response and event-driven patterns are supported.

A10 Use Test.createTestingModule({providers:

Optimization

Use Test.createTestingModule({providers: [MyService, {provide: Repo, useValue: mockRepo}]}).compile(). This creates a mini DI container. Call module.get(MyService) to retrieve the instance. Jest spies and mocks replace real dependencies.